

## 第20回 HPを守る:カプセル化

### 今回のゴール

- フィールドを `private` にして、クラスの外から直接変更できないようにできる。
- getter(値を返すメソッド)を書いて、外から値を読めるようにできる。
- 「HP は 0 未満にならない」のようなルールを、クラスのメソッドの中にまとめられる。

### 前回とのつながり

第19回の `Character` は、`Main` から `hero.hp = 999;` のように、HP を自由に書きかえられた。第16回から残っていた「HP がマイナスになる」問題もそのまま。今回は、HP を変えられる場所を `Character` のメソッドだけに限定して、ルールを守らせる。

### 実習時のフォルダ構成

```
ワークスペース
├─ java20_sample
│   └─ src
│       ├── Character.java
│       └─ Main.java
└─ java20_exercise
    └─ src
        ├── Character.java
        └─ Main.java
```

- 配布ファイル: `プログラム/第20回/sample/`、`プログラム/第20回/exercise/`
- 解答例: `プログラム/第20回/answer/`

### 講義内容

#### 1. 何が困るのか

第19回のコードでは、次のような書き方ができてしまう。

```
hero.hp = 999;           // 最大 HP が 30なのに 999 になる
enemy.hp = enemy.hp - 50; // HP がマイナスになる
```

- どちらもエラーにならず、実行できてしまう。バグの原因を探るとき、HP を変えている場所を `Main` 全体から探すことになる。
- 「HP は 0～最大 HP の間」というルールを、`Character` クラスの中だけで守らせたい。

## 2. private と public

| 修飾子                  | 使える場所     | 主に付けるもの           |
|----------------------|-----------|-------------------|
| <code>private</code> | そのクラスの中だけ | フィールド             |
| <code>public</code>  | どのクラスからでも | コンストラクタ、外から使うメソッド |

```
private int hp;                // Main から hero.hp とは書けなくなる

public void damage(int amount) {
    hp -= amount;
    if (hp < 0) {
        hp = 0;
    }
}
```

- HP を変える方法は、`damage` と `heal` だけになる。ルール(0 未満にしない、最大を超えない)は、その 2 つのメソッドの中に書けばよい。
- このように、データとルールをクラスの中に閉じ込めることを **カプセル化** という。

## 3. getter と setter

`private` にすると、外から値を読むこともできなくなる。読むためのメソッドを用意する。

```
public int getHp() {
    return hp;
}
```

- 値を返すメソッドを **getter** という。名前は `get` + フィールド名(先頭を大文字)にする習慣がある。
- 値を設定するメソッドは **setter**(例: `setPower(int power)`)という。ただし、何でも自由に設定できる `setter` を作ると、`private` にした意味がなくなる。今回の `Character` では、`setter` の代わりに `damage`・`heal`・`powerUp` のように「意味のある変え方」だけを用意している。

## 4. final: 一度決めたら変えない

```
private final String name;
private final int maxHp;
```

- `final` を付けたフィールドは、コンストラクタで 1 回だけ値を入れられる。その後は変更できない(変更しようとするコンパイルエラー)。
- 名前や最大 HP のように、バトル中に変わらない値に付ける。

## 5. サンプルを読む

プログラム/第20回/sample/Character.java

```
public class Character {
    // フィールドは private にして、クラスの外から直接変更できないようにする
```

```

private final String name;
private int hp;
private final int maxHp;
private int power;

public Character(String name, int hp, int power) {
    this.name = name;
    this.hp = hp;
    this.maxHp = hp;
    this.power = power;
}

// getter: フィールドの値を外から読むためのメソッド
public String getName() {
    return name;
}

public int getHp() {
    return hp;
}

public int getPower() {
    return power;
}

// 名前と HP を表示する
public void showStatus() {
    System.out.println(name + " HP:" + hp + "/" + maxHp);
}

// target に攻撃する (3 ターンごとに会心の一撃で 1.5 倍)
public void attack(Character target, int turn) {
    int damage = power;
    if (turn % 3 == 0) {
        System.out.println("会心の一撃!");
        damage = (int) (power * 1.5);
    }
    System.out.println(name + "の攻撃! " + target.getName() + "に" + damage + "のダメージ");
    target.damage(damage);
}

// ダメージを受ける (HP は 0 より小さくならない)
public void damage(int amount) {
    hp -= amount;
    if (hp < 0) {
        hp = 0;
    }
}

// HP を回復する (最大 HP を超えない)
public void heal(int amount) {
    hp += amount;
    if (hp > maxHp) {
        hp = maxHp;
    }
}

```

```

// HP が残っていれば true
public boolean isAlive() {
    return hp > 0;
}

// 攻撃力を amount だけ上げる
public void powerUp(int amount) {
    power += amount;
}
}

```

- `attack` の中でも、相手の HP を直接減らさず `target.damage(damage)` を呼んでいる。HP を減らすルールが `damage` の 1 か所にまとまる。
- `powerUp` は第19回の演習で作った「ためる」の処理。

プログラム/第20回/sample/Main.java では、`hero.name` や `hero.hp` と書いていたところが `hero.getName()`、`hero.getHp()` に変わった。また、最後に両者のステータスを表示している。

```
System.out.println(hero.getName() + "は薬草を使った！ HPが" + hero.getHp() + "になった");
```

## 6. 実行する前に予想する

1. を 4 回入力する(薬草は使わない)。
- 最後の勇者の HP は、第16回のようにマイナスになるか。

## 7. 実行結果(最後の部分)

```

--- ターン4 ---
勇者 HP:6/30
ゴブリン HP:12/40
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ ゴブリンに8のダメージ
ゴブリンの攻撃！ 勇者に7のダメージ

勇者 HP:0/30
ゴブリン HP:4/40
勇者はたおれてしまった...

```

- `6 - 7` は `-1` だが、`damage` のルールによって HP は `0` になっている。

## 8. エラーを体験する

1. `Main.java` の `while` の前に、`hero.hp = 999;` と 1 行書いて保存する。

2. `hp` は `private` なので使えない、というエラーになることを確認する(Pleiades では「不可視」と表示されることがある)。
3. その行を消す。次に `Character.java` の `heal` の中に `maxHp = 50;` と書いて保存し、`final` のフィールドには代入できないエラーを確認する。確認したら消す。

## 演習1 サンプルを実行する

`java20_sample` でサンプルを実行し、薬草を使っても HP が 30 を超えないこと、HP が 0 未満にならないことを確かめる。

## 演習2 Character クラスのルールを完成させる

`java20_exercise` を作り、`プログラム/第20回/exercise/` の 2 ファイルを入れる。

### 演習2-1 HP を 0 未満にしない

- まず 1 だけを入力して実行し、最後に勇者の HP がマイナスで表示されることを確認する。
- `damage` メソッドに、HP が 0 より小さくなったら 0 にする処理を書く。

### 演習2-2 getMaxHp を作って使う

- 最大 HP を返す `getMaxHp()` メソッドを作る。
- 薬草を使ったときの表示を、`勇者は薬草を使った！ HPが21/30になった` の形に変える。

### 演習2-3 攻撃力の上限を決める

- 「ためる」を何回も使うと、攻撃力がどこまでも上がってしまう。`powerUp` で、攻撃力が 20 を超えたら 20 にする。
- `Main` は変更せずにルールを追加できることを確認する。

完成条件(1、1、1、2 と入力したとき、4 ターン目の薬草の表示)：

```
コマンドを選ぶ > 2
勇者は薬草を使った！ HPが21/30になった
```

「ためる」を 4 回以上続けても、攻撃力が 20 で止まることも確認する。

### 発展(任意)

- 敵の攻撃力を一時的に下げる `powerDown(int amount)` を作る。攻撃力は 1 より小さくならないようにする。
- `showStatus` に攻撃力も表示する。

## 確認問題

問1 出力は何か(`Character` はサンプルと同じ)。

```
Character c = new Character("勇者", 30, 8);
c.damage(50);
c.heal(10);
System.out.println(c.getHp());
```

問2 `Main` クラスに次のように書いた。コンパイルできるか。

```
Character c = new Character("勇者", 30, 8);
c.hp = 999;
```

問3 次のコードはコンパイルできるか。

```
final int maxHerb = 3;
maxHerb = 5;
```

### ■ 答え

- 問1: 10 (`damage(50)` で HP は 0 で止まる。そこから 10 回復する)
- 問2: できない (`hp` は `private` なので、`Character` の外からは使えない)
- 問3: できない (`final` を付けた変数には、もう一度代入できない)

## 振り返り

- フィールドを `private` にする理由を、バトルの例で説明できるか。
- `getter` を自分で書けたか。
- `Main` を変えずに、`Character` の中だけでルールを追加できたか。

## 次回へのつながり

勇者とゴブリンは、同じ `Character` クラスなので、ゴブリンも会心の一撃を出してしまう。また、薬草の数 `herbCount` は勇者だけが持つ情報なのに、`Main` に置かれている。次回は、勇者と敵で違う部分を持てるようにする。